z-plus: Build Specification

Oct 9, 2026 · @Anuj

Overview

z-plus scans a folder of Python submissions for one coding problem and reports which ones are copies of each other, even after renaming, reordering and padding. You two write every core algorithm yourselves, with no plagiarism libraries.

The problem

A student copies a classmate's solution and disguises it: renames variables, rewrites comments, shuffles independent lines, adds dead code. A plain text diff misses this. z-plus compares the *structure* of the code instead of its text.

Input and output

- **Input:** a folder `submissions/<problem>/<author>.py`, plus an optional `template.py` (starter code given to everyone, which must be ignored).
- **Output:** a ranked list of suspicious pairs with a score from 0 to 1, groups of authors who copied from each other (rings), and the matching regions of each pair. Formats: terminal text, JSON, and a self-contained HTML report.

Goals

1. Implement every core algorithm (rolling hash, winnowing, inverted index, greedy tiling, union-find) yourselves.
2. Measure accuracy with your own disguise generator, not by eyeballing examples.
3. Make every flag explainable: show which code regions matched.
4. Scan 200 submissions of about 100 lines each in under 10 seconds on a laptop (a goal to check, not a result).

Non-goals

- Detecting *semantic* clones (a different algorithm that gives the same output).
- Languages other than Python.
- A web app or login system.

- Judging intent. A flag is a lead for a human reviewer, never a verdict.

Tech stack

- Python 3.10 or newer.
- Standard library only for the engine: `tokenize`, `ast`, `hashlib`, `collections`, `dataclasses`, `argparse`, `json`, `html`. `difflib` is used only for the baseline comparison in evaluation.
- `pytest` for tests, `matplotlib` for evaluation plots (Person B only), git and GitHub for collaboration.

Key terms

Term	Meaning
Token	One lexical unit of code: a keyword, operator, name, number, string or indent marker
Normalization	Rewriting tokens so cosmetic changes vanish: every variable name becomes `ID`, every number `NUM`, comments dropped
k-gram	A window of k consecutive tokens
Fingerprint	A (hash, position) pair that stands for one selected k-gram
Winnowing	Keeping only the smallest hash in each sliding window of w k-gram hashes, so each file keeps a small, guaranteed-to-overlap sample
Inverted index	A map from a fingerprint hash to the list of submissions that contain it
Candidate pair	Two submissions that share enough fingerprints to deserve a closer look
Tiling	Covering two token sequences with the longest shared unmarked runs, used to verify and highlight matches
Cluster (ring)	A group of submissions linked by pairwise matches above the threshold

Work division

Each person owns six modules worth about 11 focused days, plus four shared tasks. Person A owns the front half of the pipeline (files to candidate pairs). Person B owns the back half (candidate pairs to verdicts, plus the proof that it works). Decide between you who is A and who is B.

Module ownership

ID	Module	Owner	Delivers	Days (approx.)
A1	Loader and normalizer	A	`load_submissions`, `normalize`	2
A2	Rolling hash of k-grams	A	`kgram_hashes`	1.5
A3	Winnowing	A	`winnow`	2
A4	Inverted index and candidate pairs	A	`build_index`, `candidate_pairs`, template filtering	2
A5	Dataset layout and ground truth	A	`data/` structure, `tests.json`, `check_solution`, `ground_truth`	1.5
A6	AST structural hashing	A	`ast_similarity`	2
B1	Similarity scoring	B	`score_pair`, `combine`	1.5
B2	Greedy string tiling	B	`greedy_tiling`	2.5
B3	Union-find clustering	B	`UnionFind`, `cluster`	1
B4	Mutation generator	B	`mutate`, `make_copies`	2.5
B5	Evaluation harness	B	`evaluate`, `sweep`, plots	2
B6	CLI and reports	B	`zplus scan`, JSON and HTML report	1.5

Totals: Person A 11 days, Person B 11 days.

Shared tasks

ID	Task	Who	Days each
S1	Repo setup, data contracts, empty module stubs	Both	1
S2	Write 20 independent solutions each (4 per problem, 5 problems)	Both, in parallel	1
S3	Integration: wire `run()`, fix interface mismatches, first full run	Both	1

ID	Task	Who	Days each
S4	Interview preparation and mock interviews	Both	1

Each person ends up with 15 days of work.

Why this split is fair

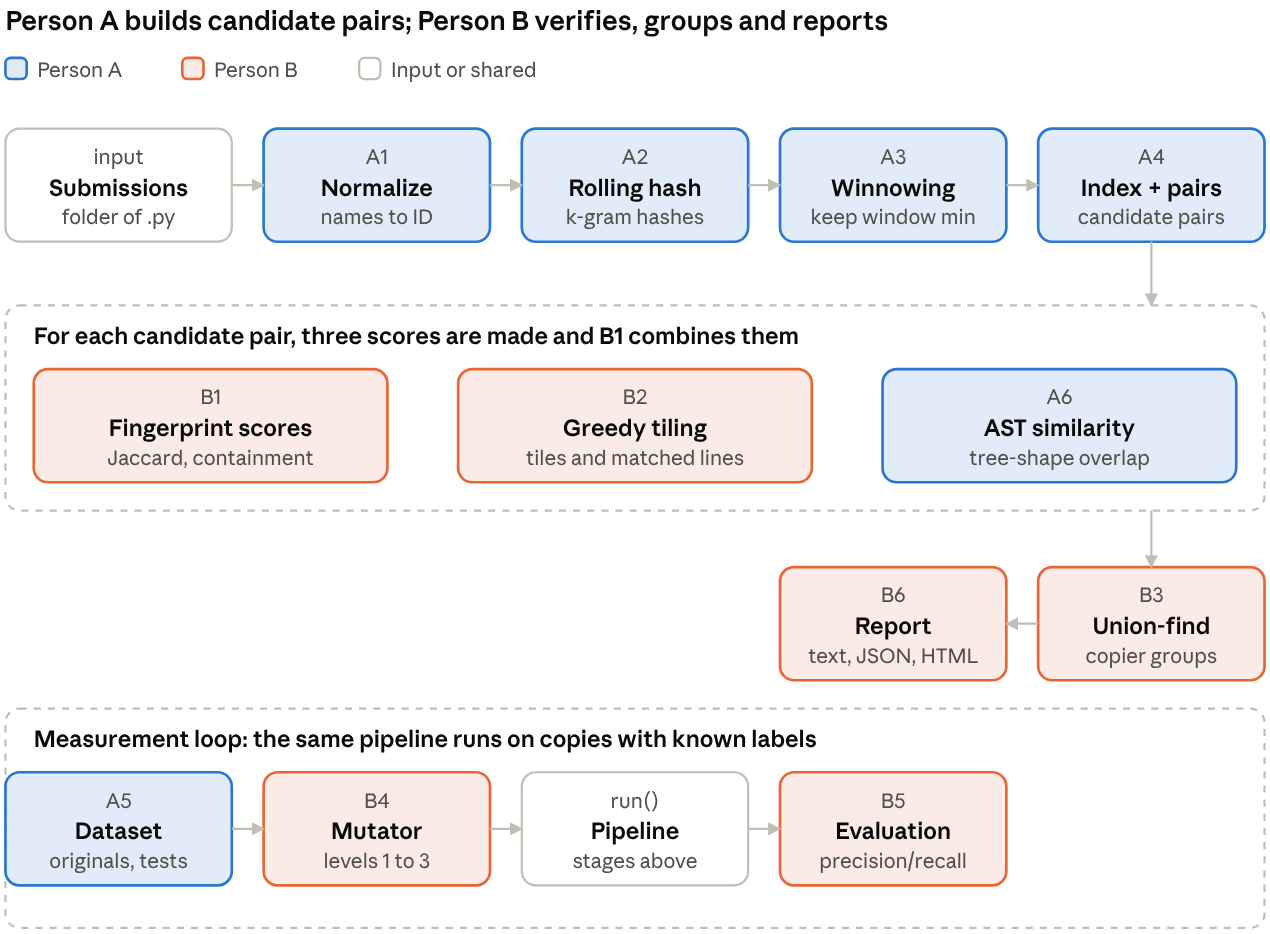
Each person gets the same mix of work, so neither ends up with only the easy or only the hard parts:

- **One hard algorithm:** A has winnowing with the rolling hash, B has greedy string tiling.
- **One data-structure module:** A has the inverted index, B has union-find.
- **One data-generation module:** A has the dataset and ground truth, B has the mutation generator.
- **One module that shows off the result:** A has AST hashing, B has the CLI and HTML report.
- **One module that decides quality:** A has normalization (what counts as "the same"), B has the evaluation harness (how well it works).

Because each person must be able to explain the other's half in an interview, the cross-review rule in the collaboration section is part of the split, not an extra.

Architecture and repository layout

Person A's four stages turn files into candidate pairs; Person B's stages verify those pairs, group them and report, while a separate loop measures accuracy.



pipeline architecture · 12 modules, 2 owners

Read it top to bottom: A1 to A4 run once per problem, the dashed middle band runs once per candidate pair, and the bottom loop feeds planted copies with known labels through the same pipeline.

Repository layout

```
z-plus/
  README.md           # how it works, per module, in your own words
  pyproject.toml      # project metadata, pytest settings
  zplus/
    __init__.py
    __main__.py       # python -m zplus -> cli.main()
    types.py          # shared dataclasses (S1)
    errors.py         # ZplusError
    loader.py         # A1: load_submissions
    normalize.py      # A1: normalize
    rollinghash.py    # A2: kgram_hashes
```

```
    winnow.py          # A3: winnow
    index.py           # A4: template_hashes, build_index, candidate_pairs
    dataset.py         # A5: ground_truth, check_solution, validate_dataset
    astsim.py          # A6: ast_similarity
    scoring.py         # B1: score_pair, combine
    tiling.py          # B2: greedy_tiling
    cluster.py         # B3: UnionFind, cluster
    mutate.py          # B4: mutate, make_copies
    evaluate.py        # B5: evaluate, sweep, plots
    report.py          # B6: text, JSON and HTML writers
    cli.py             # B6: argparse subcommands scan and eval
    pipeline.py        # S3: run()
  data/                # A5: problems, originals, planted copies
  tests/
    test_<module>.py   # one file per module
    fixtures/          # small hand-made inputs
  results/             # B5 output
  .github/workflows/test.yml
```

What can start on day 2

Every module below can be built and tested against hand-made inputs, so nobody waits.

Module	Can start with	Waits for
A1 normalize	Any `.py` file	Nothing
A2 rolling hash	A hand-made token list	Nothing (uses the `Token` type)
A5 dataset	Your own problems	Nothing
B1 scoring	Hand-made fingerprint lists	Nothing
B2 tiling	Hand-made token lists	Nothing
B3 clustering	Hand-made `PairResult` objects	Nothing
A3 winnowing	Output of A2	A2
A4 index	Output of A3	A3
A6 AST hashing	Any `.py` file	Nothing, but wired in at S3

Module	Can start with	Waits for
B4 mutator	Any original file	A5's `check_solution` for its safety net
B5 evaluation	Hand-made predictions	`run()` and B4's copies for real runs
B6 CLI and report	A fake `Report` object	`run()` for real runs

Shared data contracts

Both people code against the types and signatures below from day 1, so nobody waits for the other. Write them into `zplus/types.py` and the stub files during task S1, and change them only by agreement.

Data types

```
# zplus/types.py
from dataclasses import dataclass

@dataclass(frozen=True)
class Token:
    kind: str   # "KW", "BUILTIN", "ID", "NUM", "STR", "OP", "NEWLINE",
"INDENT", "DEDENT"
    text: str   # keywords, builtins, operators as written; "ID", "NUM",
"STR" otherwise
    line: int   # 1-based line in the ORIGINAL file, used for highlighting

@dataclass
class Submission:
    sid: str        # unique id, e.g. "p1/alice"
    author: str     # file stem, e.g. "alice"
    problem: str    # folder name, e.g. "p1"
    path: str
    source: str     # raw text
    tokens: list[Token]

@dataclass(frozen=True)
class Fingerprint:
    hash: int       # 0 <= hash < 2**61 - 1
    pos: int        # index of the k-gram's first token in Submission.tokens
```

```
@dataclass(frozen=True)
class Candidate:
    a: str          # sid; a < b alphabetically so each pair appears once
    b: str
    shared: int     # number of distinct fingerprint hashes the two share

@dataclass(frozen=True)
class Match:
    a_start: int    # token index in submission a
    b_start: int    # token index in submission b
    length: int     # number of tokens

@dataclass
class Scores:
    jaccard: float
    containment: float        # max(|A&B|/|A|, |A&B|/|B|) on fingerprint
hashes
    tiling: float | None      # from B2, None if not run
    ast: float | None         # from A6, None if disabled
    final: float              # decided in B1 (combine)

@dataclass
class PairResult:
    a: str
    b: str
    scores: Scores
    matches: list[Match]

@dataclass
class Config:
    k: int = 6                # k-gram length in tokens
    w: int = 4                # winnowing window, in k-grams
    min_shared: int = 3       # A4: fewest shared hashes to become a
candidate
    max_df: float = 0.3       # A4: drop hashes found in more than 30% of
submissions
    min_match: int = 8        # B2: shortest tile, in tokens
    threshold: float = 0.6    # B3 and B6: score at which a pair is flagged
    use_ast: bool = True      # A6 on or off
```

Function signatures

Owner	Signature	Notes
A1	`load_submissions(folder: str) -> list[Submission]`	Skips unreadable or unparsable files with a logged warning
A1	`normalize(source: str, keep_names: bool = False) -> list[Token]`	Raises `ZplusError` on unrecoverable syntax
A2	`kgram_hashes(tokens: list[Token], k: int) -> list[tuple[int, int]]`	List of (hash, pos), one per k-gram, in order
A3	`winnow(hashes: list[tuple[int, int]], w: int) -> list[Fingerprint]`	Keeps the minimum of each window
A4	`build_index(fps: dict[str, list[Fingerprint]], max_df: float, ignore: frozenset[int] = frozenset()) -> dict[int, set[str]]`	Maps hash to the sids that contain it
A4	`candidate_pairs(index: dict[int, set[str]], min_shared: int) -> list[Candidate]`	Called once per problem; template_hashes(template_path, cfg) -> set[int] supplies ignore
A5	`ground_truth(folder: str) -> set[frozenset[str]]`	Unordered pairs known to be copies
A5	`check_solution(path: str, tests_path: str) -> bool`	Runs the file's `solve` on the test cases in a subprocess
A6	`ast_similarity(src_a: str, src_b: str, min_nodes: int = 8) -> float`	0 to 1
B1	`score_pair(fa: list[Fingerprint], fb: list[Fingerprint]) -> Scores`	`tiling` and `ast` start as None
B1	`combine(s: Scores) -> float`	Sets `final`

Owner	Signature	Notes
B2	`greedy_tiling(ta: list[Token], tb: list[Token], min_match: int) -> list[Match]`	Non-overlapping tiles
B3	`cluster(pairs: list[PairResult], threshold: float) -> list[set[str]]`	Groups of size 2 or more
B4	`mutate(source: str, level: int, seed: int) -> str`	Level 1, 2 or 3
B4	`make_copies(folder: str, per_problem: int, seed: int) -> None`	Writes planted copies into `data/`
B5	`evaluate(folder: str, cfg: Config) -> Metrics`	Precision, recall, F1
B6	`run(folder: str, cfg: Config, template: str | None) -> Report`	The full pipeline

The pipeline in code

B writes `run()` in task S3, calling A's functions like this. The helpers group_by_problem and tiling_score (the formula from B2) are small functions you add in the same task:

```
def run(folder, cfg, template=None):
    subs = load_submissions(folder)                                  # A1
    tokens = {s.sid: s.tokens for s in subs}
    src = {s.sid: s.source for s in subs}
    ignore = template_hashes(template, cfg) if template else frozenset()   #
A4
    results = []
    for problem, group in group_by_problem(subs):
        fps = {s.sid: winnow(kgram_hashes(s.tokens, cfg.k), cfg.w)  # A2, A3
               for s in group}
        index = build_index(fps, cfg.max_df, ignore)                 # A4
        for c in candidate_pairs(index, cfg.min_shared):             # A4
            sc = score_pair(fps[c.a], fps[c.b])                      # B1
            matches = greedy_tiling(tokens[c.a], tokens[c.b], cfg.min_match)
# B2
```

```
            sc.tiling = tiling_score(matches, len(tokens[c.a]),
len(tokens[c.b]))
            if cfg.use_ast:
                sc.ast = ast_similarity(src[c.a], src[c.b])         # A6
            sc.final = combine(sc)                                  # B1
            results.append(PairResult(c.a, c.b, sc, matches))
    groups = cluster(results, cfg.threshold)                        # B3
    return Report(results, groups)                                  # B6
```

Rules every module follows

- **Pure and deterministic:** the same input gives the same output. Anything random takes an explicit `seed`.
- **No printing in the engine:** use the `logging` module. Only B6 prints.
- **Never use Python's built-in `hash()` on strings.** It is randomized per process, so fingerprints would change between runs. Use the stable hash defined in A2.
- **Token indices are 0-based, line numbers are 1-based.**
- **Type hints and a docstring on every public function,** including its time complexity.

Ground-truth naming convention (A5 and B4)

- Independent original: `<author>.py`, for example `alice.py`.
- Planted copy: `copy<n>_L<level>_of_<author>.py`, for example `copy2_L3_of_alice.py`.
- Two files are a **true positive pair** if they share the same root original (an original and its copies, or two copies of the same original). Every other pair is a negative.
- Each problem folder holds a `tests.json`: `{"function": "solve", "cases": [{"args": [..], "expected": ..}]}`. Every file must define that function.

Person A modules

Person A builds the front half: raw files in, candidate pairs out. Every module below lists what it does, how, what can go wrong, how to test it, and when it counts as done.

A1. Loader and normalizer (2 days)

Purpose: turn raw files into token lists in which cosmetic edits disappear.

Steps

1. `load_submissions(folder)`: walk `folder/<problem>/*.py` (a flat folder also works, with the folder name as the problem). Read each file as UTF-8 with `errors="replace"`. Set `sid = "<problem>/<stem>"`.
2. `normalize(source)`: run `tokenize.generate_tokens(io.StringIO(source).readline)` and map every token with the table below.
3. Remove docstrings and bare string statements: a STRING token that stands alone on its logical line is dropped.
4. Return the list of `Token(kind, text, line)`, keeping each token's original line number.

Python token	Becomes	Why
NAME that is a keyword (`keyword.iskeyword`)	`KW`, text kept	Keywords carry the program's shape
NAME found in `dir(builtins)` (`len`, `range`, `print`)	`BUILTIN`, text kept	Using `sorted` versus `min` is real information
Any other NAME	`ID`, text `ID`	Renaming must not matter
NUMBER	`NUM`, text `NUM`	Changing a constant must not matter
STRING	`STR`, text `STR`	Changing a message must not matter
OP	`OP`, text kept	`+` versus `-` matters
NEWLINE	`NEWLINE`	Marks the end of a statement
INDENT, DEDENT	kept	Marks block structure
COMMENT, NL, ENCODING, ENDMARKER	dropped	NL is a blank or continuation line, so wrapping and blank lines never matter

Edge cases

- Empty file: return `[]`.
- Python 3.12 splits f-strings into `FSTRING_START`, `FSTRING_MIDDLE`, `FSTRING_END` and nested tokens. Collapse the whole f-string into one `STR` using a depth counter.

- `IndentationError`, `TabError`, `tokenize.TokenError`: log a warning and skip the file in `load_submissions`.
- Windows line endings, a byte-order mark, and tabs versus spaces must give the same tokens.
- Files over 20,000 tokens: log a warning (they slow tiling in B2).

Trade-off to remember: attribute and method names (`x.append`) become `ID`, so `append` and `extend` look the same. That raises recall and lowers precision. Be ready to defend it.

Complexity: O(n) in the number of characters.

Tests

- Renaming every variable gives an identical token list.
- Adding comments, blank lines and docstrings gives an identical token list.
- Wrapping a long expression across lines gives an identical token list.
- An f-string becomes exactly one `STR`.
- A file with a syntax error is skipped, not a crash.
- `token.line` points at the correct original line.

Done when: all tests pass, and a short README paragraph explains each mapping choice in your own words.

A2. Rolling hash of k-grams (1.5 days)

Purpose: turn the token list into one hash per k-gram in O(n) time.

Steps

1. Convert each token to an integer with a stable hash of its text: `int.from_bytes(hashlib.blake2b(text.encode(), digest_size=8).digest(), "big") % M`. Cache the results in a dict.
2. Fix the modulus `M = 2**61 - 1` (a Mersenne prime) and the base `B = 911382323`. Fixed constants keep results reproducible.
3. Hash the first k-gram directly, then roll the window one token at a time. Precompute `B^(k-1) mod M` once.
4. Return `[(hash, pos)]` with `pos` as the index of the k-gram's first token.

```
H_i = \sum_{j=0}^{k-1} t_{i+j} \, B^{\,k-1-j} \bmod M

H_{i+1} = \big( (H_i - t_i \, B^{\,k-1}) \, B + t_{i+k} \big) \bmod M
```

Edge cases

- Fewer than k tokens: return `[]`.
- `k < 1`: raise `ValueError`.
- Python's `%` already returns a non-negative result, so the subtraction in the roll is safe.
- Never use built-in `hash()` on strings. It is randomized per process.

Complexity: O(n) time and space. Recomputing every window from scratch would be O(nk). With a 61-bit modulus the chance that two different k-grams collide is about 1 in 2.3 quintillion, which is negligible at this scale.

Tests

- On 1,000 random token sequences, the rolling result equals a naive recomputation of every window.
- The same k-gram at two positions gives the same hash.
- Two runs in separate Python processes give identical output.
- `len(result) == n - k + 1`.

Done when: all tests pass and you can derive the roll formula on a whiteboard without notes.

A3. Winnowing (2 days)

Purpose: shrink the hash list to a small sample that is still guaranteed to overlap between copies. This follows Schleimer, Wilkerson and Aiken, "Winnowing: Local Algorithms for Document Fingerprinting" (2003).

Algorithm: slide a window of w consecutive hashes along the list. In each window pick the smallest hash (the rightmost one on ties). Record `(hash, pos)` unless it is the same position as the last one recorded.

Guarantee: any shared token run of length at least `w + k - 1` produces at least one shared fingerprint. Matches shorter than `k` tokens are never found. About `2 / (w + 1)` of all k-grams are kept.

Steps

1. If the list is empty, return `[]`. If it is shorter than w, return the single minimum (rightmost on ties).
2. Keep a deque of indices whose hashes increase from front to back. For each new index `i`, pop from the back while `hashes[back] >= hashes[i]`, then push `i`. Pop from the front while `front <= i - w`.
3. Once `i >= w - 1`, the front of the deque is the window minimum. Emit it if its position differs from the last emitted one.
4. Return fingerprints sorted by `pos`.

Edge cases

- Many equal hashes (a repeated loop body): the rightmost rule keeps the result stable.
- `w == 1`: every k-gram is kept.
- `w < 1`: raise `ValueError`.

Complexity: O(n) time with the deque. A plain scan of each window is O(nw).

Why not "keep hashes divisible by p"? That rule can select nothing across a long stretch, so a long copied block might leave no fingerprint at all. Winnowing's guarantee closes that hole.

Tests

- The deque version equals a brute-force O(nw) version on random inputs.
- Guarantee test: plant a shared block of at least `w + k - 1` tokens inside two otherwise random sequences, run 1,000 trials, and require at least one shared fingerprint hash every time.
- Density: the kept fraction is close to `2 / (w + 1)` on long random input.
- Positions are strictly increasing, with no duplicates.

Done when: the guarantee test passes 1,000 out of 1,000 and you can explain in two minutes why the guarantee holds.

A4. Inverted index and candidate pairs (2 days)

Purpose: find the few pairs worth comparing without checking every pair of files.

Steps

1. `template_hashes(template_path, cfg) -> set[int]`: normalize, hash and winnow the starter file. These hashes are ignored everywhere, so shared starter code never counts as copying.

2. `build_index(fps, max_df, ignore=frozenset())`: build `dict[hash, set[sid]]` for **one problem** at a time (`run()` groups submissions by problem). Skip any hash in `ignore`.
3. Drop "boilerplate" hashes, the way a search engine drops stop words. With N submissions in the problem, drop a hash that appears in more than `max(3, ceil(max_df * N))` of them. The floor of 3 stops small folders from losing real matches.
4. `candidate_pairs(index, min_shared)`: for each posting list, sort the sids and add 1 to a counter for every pair `(a, b)` with `a < b`. Return pairs with a count of at least `min_shared`, sorted by count, highest first.

Edge cases

- A submission with zero fingerprints (too short) never appears in the index. Record it so the report can say "too short to check".
- Two identical files share every hash.
- One submission in a problem gives no pairs.
- A very long posting list makes the pair loop explode. That is exactly why step 3 exists.

Complexity: building the index is O(total fingerprints). Counting pairs costs the sum over hashes of `|posting list|^2`, which stays small once common hashes are dropped. Comparing all pairs directly would be O(N^2) comparisons of whole files.

Tests

- A hand-made index with known pair counts.
- A boilerplate hash is dropped once N is large enough, and kept when N is tiny.
- A submission that is exactly the template produces no candidates.
- Every returned pair has `a < b` and no pair repeats.
- A synthetic run with 1,000 submissions finishes quickly (measure and note the time in the README).

Done when: tests pass and you can explain why the index beats all-pairs comparison and what `max_df` trades away.

A5. Dataset layout and ground truth (1.5 days)

Purpose: build the data that all accuracy claims depend on. A5 owns the layout and the checking tools. Both of you write the solutions in task S2.

```
data/
  p1_merge_intervals/
    problem.md            # statement, 5 to 10 lines
    tests.json            # 5 to 8 cases, including edge cases
    alice.py, bob.py ...  # independent originals
    copy1_L1_of_alice.py  # planted copies (written by B4)
  p2_.../
```

Choosing problems: five problems whose honest solutions run 15 to 40 lines and allow at least two different approaches. Suggested: merge intervals, valid parentheses with several bracket types, top-k frequent words, spiral matrix traversal, number of islands on a grid. Write the solutions yourselves; do not copy from the web.

Target size: 8 originals per problem (40 total) plus 6 planted copies per problem (2 per level). That makes about 70 files, about 30 positive pairs and about 425 negative pairs.

Steps

1. Write `problem.md` and `tests.json` for each problem.
2. `check_solution(path, tests_path)`: run the file in a **subprocess** (`sys.executable`, 5-second timeout), import it with `importlib`, call `solve` on each case, compare with `expected`. Never `exec` submissions inside your own process.
3. `ground_truth(folder)`: parse names with the regex `^copy\d+_L([123])_of_(.+)\.py$`. Each file's root is its origin's author (an original is its own root). All unordered pairs inside one root group are positives.
4. `validate_dataset(folder)`: every file passes `check_solution`, every copy's origin exists, no two files are byte-identical.
5. Write `data/README.md` describing the convention.

Hard negatives: in S2, each person deliberately writes at least one solution per problem that uses the same approach as the other person's, in their own words. These look alike but are honest, so they test false positives.

Edge cases: a solution that prints instead of returning fails the check. Top-level code must sit under `if __name__ == "__main__":`. Infinite loops are cut off by the timeout. Use forward slashes in sids on every OS.

Tests

- `ground_truth` returns the expected set on a toy folder with known copies.

- `check_solution` accepts a correct file, rejects a wrong one, and rejects an infinite loop within the timeout.

Done when: `validate_dataset` passes on all five problems and the README is written.

A6. AST structural hashing (2 days)

Purpose: a second opinion that survives tricks aimed at token sequences, such as reordering independent statements.

Algorithm

1. Parse with `ast.parse`.
2. Walk the tree bottom-up and give every node a structure hash: a stable hash (blake2b, not `hash()`) of the string `NodeType(childHash1,childHash2,...)`. Ignore identifier names and constant values. Keep operator node types, so `Add` and `Sub` differ. Skip `Load`, `Store` and `Del` context nodes.
3. Compute each subtree's size in nodes. Keep only subtrees with at least `min_nodes` nodes (default 8) so tiny fragments do not count.
4. Put the kept hashes in a multiset (`collections.Counter`).
5. `ast_similarity(a, b)` is the overlap coefficient of the two multisets: the sum of `min(countA, countB)` divided by the smaller total size.

Why it helps: swapping two independent statements changes only their parent's hash. Every statement subtree below keeps its own hash and still matches. Adding dead code adds new subtrees but removes none, and the overlap coefficient stays high where plain Jaccard would fall.

Limits to state honestly: a `for` to `while` rewrite changes node types, so those subtrees do not match. A tiny file inside a big one can score 1.0, which is why the final score in B1 also looks at tiling. As an optional improvement, canonicalize `x += 1` into `x = x + 1` before hashing.

Edge cases

- Syntax error: return 0.0 and log a warning.
- No subtree reaches `min_nodes`: return 0.0.
- Deeply nested code: raise the recursion limit or walk iteratively.
- Rely only on node class names, so it works across Python versions.

Complexity: O(number of AST nodes).

Tests

- Identical files give 1.0.
- Renamed variables give 1.0.
- Two reordered independent statements give at least 0.8.
- Two unrelated solutions to the same problem score clearly lower. Record the numbers you see.
- A syntax error gives 0.0.

Done when: tests pass and A6′s score is wired into `run()` behind `cfg.use_ast`.

What Person A needs from Person B

- The `Token` and `Scores` types (agreed in S1).
- B4′s planted copies, to test the whole front half on realistic data in week 3.
- A first `run()` in S3, so A′s stages can be checked end to end.

Person B modules

Person B builds the back half: candidate pairs in, verified verdicts and evidence out, plus the proof that it all works. Each module has the same layout as in Person A′s section.

B1. Similarity scoring (1.5 days)

Purpose: turn two fingerprint lists into scores, and combine all available scores into one final number.

Steps

1. Reduce each fingerprint list to a set of hashes, `F_A` and `F_B`.
2. Jaccard similarity: shared hashes over all distinct hashes.
3. Containment: the larger of the two shares of each file that is found in the other. This catches a small solution pasted inside a bigger program.
4. `combine(scores)`: let `primary = tiling` if B2 ran, else `containment`. Then `final = max(primary, ast)` when the AST score exists, else `primary`.

```
J(A,B) = \frac{|F_A \cap F_B|}{|F_A \cup F_B|}

C(A,B) = \max\left( \frac{|F_A \cap F_B|}{|F_A|}, \; \frac{|F_A \cap F_B|}
{|F_B|} \right)
```

Why `max` and not an average: a cheater who defeats one view (token order) is still caught by the other (tree structure). The cost is more false positives. B5 compares `max` against the average, so you can defend the choice with numbers.

Edge cases

- Empty fingerprint sets: return 0.0 for every score, never divide by zero.
- Identical sets: 1.0.
- Very different file sizes: containment stays meaningful while Jaccard drops.

Complexity: O(|F_A| + |F_B|) with Python sets.

Tests

- Hand-made sets with known scores.
- Identical sets and disjoint sets.
- A subset gives containment 1.0 and Jaccard below 1.0.
- Empty input.
- `combine` with `tiling=None` and with `ast=None`.

Done when: all tests pass and the README states the formulas and why `final` is built this way.

B2. Greedy string tiling (2.5 days)

Purpose: verify a candidate pair and find exactly which regions match. This gives the coverage score and the highlighted evidence. The algorithm is Wise's Greedy String Tiling, in a simple form.

Algorithm

1. Mark every token of both files as unmarked.
2. Build a table `run[i][j]`: the number of consecutive equal, unmarked tokens starting at `a[i]` and `b[j]`. Fill it from the bottom right: `run[i][j] = run[i+1][j+1] + 1` if `a[i].text == b[j].text` and both are unmarked, else 0.
3. Find the longest run `L` in the table. If `L < min_match`, stop.
4. Take every run of length `L`. Accept each one that does not overlap a tile already accepted in this round. Mark its tokens in both files and record `Match(i, j, L)`.
5. Go back to step 2.

The coverage score is the share of tokens covered by tiles:

```
\text{tiling}(A,B) = \frac{2 \sum_{t} \text{len}(t)}{|A| + |B|}
```

To highlight, map `Match` token ranges to source lines through `Token.line`.

Why tiling and not just LCS: a longest-common-subsequence or longest-common-substring score cannot see two blocks that were moved to different places. Tiling finds every block, in any order, and never uses a token twice.

Edge cases

- Identical files: one tile covering everything.
- No shared run of `min_match` tokens: return `[]`.
- Ties between equal-length runs: break them by smallest `i`, then smallest `j`, so output is deterministic.
- Size guard: if `len(a) * len(b)` exceeds 4,000,000, log a warning, skip tiling and let `combine` fall back to containment.

Complexity: O(nm) per round and a handful of rounds, so about O(nm x tiles) worst case. Fine for files of a few hundred tokens. A faster version, Running Karp-Rabin GST, uses hashing to skip most of the table; treat it as a stretch goal.

Tests

- Identical files: one tile, tiling score 1.0.
- Unrelated files: no tiles, score 0.0.
- A block moved to a different position: found, with the correct start indices.
- Two separate shared blocks: two tiles.
- No token appears in two tiles (assert this on random inputs).
- Every tile is at least `min_match` long.
- Same input gives the same output on repeated runs.

Done when: tests pass and you can draw one worked example on paper, step by step.

B3. Union-find clustering (1 day)

Purpose: group flagged pairs into rings of students who copied from each other.

Steps

1. Implement `UnionFind` with a `parent` dict and a `size` dict.
2. `find(x)`: follow parents to the root, halving the path as you go (iterative, no recursion).

3. `union(a, b)`: attach the smaller tree under the larger (union by size).
4. `cluster(pairs, threshold)`: for every `PairResult` with `scores.final >= threshold`, union its two sids. Return the groups of size 2 or more as sets, sorted by size (largest first), then by smallest sid.

Why union-find and not BFS or DFS: edges arrive as a stream, so there is no need to build adjacency lists first. Each operation costs almost constant time, and you can add new edges later without redoing the work.

Honest limitation: if A matches B and B matches C, all three form one group even when A and C look unrelated. B6 shows each group's weakest link so a human can judge.

Edge cases

- No pairs, or a threshold above every score: return `[]`.
- A pair repeated twice, or a pair of a sid with itself: ignored.
- Sids that appear in no flagged pair never show up.

Complexity: O(E alpha(n)) in total, where alpha is the inverse Ackermann function, below 5 for any input you will ever see.

Tests

- A-B and B-C give one group of three.
- Two separate pairs give two groups.
- The threshold filters correctly.
- Property test: on 500 random graphs, the groups equal those found by a BFS-based reference.

Done when: the property test passes and you can explain union by size and path compression in under two minutes.

B4. Mutation generator (2.5 days)

Purpose: create disguised copies with known ground truth, so accuracy can be measured instead of guessed. Each level includes everything from the levels below it.

Level	Disguises applied	How
1. Cosmetic	Rename every user-defined name, drop and add comments, change spacing and blank lines	Rename with an `ast.NodeTransformer`, output with `ast.unparse` (which drops comments), then insert random comment lines
2. Light structure	Level 1, plus swap independent adjacent statements, insert dead code, flip comparisons (`a < b` to `b > a`)	Transformers on the AST
3. Structural	Level 2, plus `for i in range(n)` to a `while` loop, `x += y` to `x = x + y`, invert `if c: A else: B` to `if not c: B else: A`, reorder top-level function definitions	Transformers on the AST

Implementation rules

- **Renaming:** rename function names, arguments, assigned variables and comprehension variables consistently. Never rename built-ins, imported names, attribute names (`x.append`), keyword-argument names, or the entry function `solve`. Pick new names from a pool with a `random.Random(seed)`.
- **Independent statements:** two adjacent statements may be swapped only if both are plain assignments with no function calls, neither writes a name the other reads or writes, and neither contains `return`, `break`, `continue`, `raise`, `del`, `global` or `nonlocal`.
- **Dead code:** unused variables with fresh names, an `if False:` block with plausible statements, an unused helper function, `for _ in range(0): pass`.
- **Loop conversion:** only when the body has no `continue`, never assigns the loop variable, and the loop has no `else` clause.
- **Function reordering:** only plain top-level `def`s with no decorators and no default arguments.
- **Safety net:** after mutating, run `compile()`, then A5's `check_solution`. If it fails, retry with `seed + 1`, up to 5 times. If it still fails, skip that operation for that file and log it.

- **`make_copies(folder, per_problem, seed)`:** for each problem pick `per_problem` different originals, apply levels in the order 1, 2, 3, 1, 2, 3, and write `copy<n>_L<level>_of_<author>.py`.

Edge cases: names used inside strings (`getattr(obj, "name")`) are not renamed, so avoid them in the originals. Recursive functions work because every `Name` node is renamed together. Nested functions and lambda arguments must be renamed too. Classes are optional.

Complexity: linear in the size of the AST.

Tests

- Every mutant compiles.
- Every mutant passes `check_solution`, over all five problems and 20 different seeds.
- A level 1 mutant has the same normalized tokens as `ast.unparse(ast.parse(original))`, which proves renaming and comment changes are invisible after normalization.
- The same seed gives byte-identical output.
- Level 3 actually changes the token sequence (otherwise it tests nothing).

Done when: `make_copies` writes 30 validated copies into `data/`.

B5. Evaluation harness (2 days)

Purpose: measure accuracy, choose parameters, and produce the numbers you will quote in interviews.

Definitions

- Every pair of files inside one problem is a prediction. A pair is **flagged** if `final >= threshold`. A pair that never became a candidate counts as not flagged.
- A pair is **truly positive** if A5's `ground_truth` lists it.
- TP, FP and FN count flagged positives, flagged negatives and missed positives.

```
\text{precision} = \frac{TP}{TP + FP} \qquad \text{recall} = \frac{TP}{TP + FN} \qquad F_1 = \frac{2PR}{P + R}
```

If nothing is flagged, define precision as 1.0 and say so in the README.

Experiments (one command, `python -m zplus eval data/`, with fixed seeds)

1. **Threshold sweep** from 0.10 to 0.90 in steps of 0.05: a precision-recall curve as CSV and PNG.
2. **Parameter grid** over `k` in {3, 5, 7, 9} and `w` in {2, 4, 6}: an F1 table that picks the defaults.
3. **Recall by level:** recall at the chosen settings for planted copies of level 1, 2 and 3 separately.
4. **Ablations:** tiling only, AST only, `max` versus average in `combine`, and normalization off. The last needs an optional `keep_names` flag on A1′s `normalize`; agree on it in S1.
5. **Baseline:** `difflib.SequenceMatcher(None, text_a, text_b).ratio()` on raw source. Report recall at the precision z-plus reaches.
6. **Runtime:** time for 50, 100, 200 and 400 submissions (duplicate and mutate files to reach the size) and plot time against count.

Avoid fooling yourselves: tune parameters on problems 1 to 3 and report final numbers only on the held-out problems 4 and 5. Say this in the README.

Edge cases: a problem with no copies has no positives, so recall is undefined; skip it in recall. Report averages over problems and the pooled total.

Complexity: the grid runs the pipeline 12 times plus the sweep. Cache fingerprints per `(k, w)` so the threshold sweep reuses them.

Tests

- Metrics on a tiny hand-made confusion matrix.
- The harness on a two-file toy folder.
- Raising the threshold never raises recall (assert it on the real data).

Done when: `results/summary.md` and the plots are produced by one command, and the headline numbers are copied into the project README.

B6. CLI and reports (1.5 days)

Purpose: let anyone run z-plus on a folder and read the evidence.

```
python -m zplus scan data/p1_merge_intervals --k 6 --w 4 --threshold 0.6 \
    --template starter.py --format html --out report.html
python -m zplus eval data/ --out results/
```

Options for `scan`: `--k`, `--w`, `--threshold`, `--min-shared`, `--max-df`, `--min-match`, `--no-ast`, `--template`, `--format {text,json,html}`, `--out`, `--top N`, `--verbose`. Build it with `argparse` and subcommands.

Report contents

- Header: the configuration, the number of files read, the number skipped and why.
- A table of flagged pairs sorted by `final`: the two files, final score, tiling, AST and Jaccard.
- The groups (rings) from B3, each with its members and its weakest pair score.
- For each flagged pair, the two sources side by side with matched regions highlighted. Map each `Match` to a line range with `Token.line`.
- Files that were too short or failed to parse.

HTML rules: one self-contained file with inline CSS. Run every line of source through `html.escape`, so a submission containing `<script>` cannot inject code into the report. Use `<mark>` with a different color per tile.

JSON shape: `{config, pairs: [{a, b, scores, matches: [{a_lines: [start, end], b_lines: [start, end], tokens}]}], clusters: [[sid, ...]], skipped: [{sid, reason}]}`.

Exit codes: 0 for no flags, 1 for flags found, 2 for errors such as a missing folder.

Tests

- Run the CLI in a subprocess on a tiny fixture and check the exit code and valid JSON.
- A fixture whose source contains `<script>` produces `<script>` in the HTML.
- A snapshot test for the text output.
- A missing folder gives exit code 2 and a clear message.

Done when: a person who has never seen the project can follow the README command and get a readable report.

What Person B needs from Person A

- Fingerprints with positions and token lists with line numbers (A1 to A4).
- `check_solution` and `ground_truth` from A5, early in week 1, because B4 and B5 depend on them.
- `ast_similarity` from A6 before the full evaluation run.

Testing and evaluation plan

Every module is tested alone, then the whole pipeline is tested on fixtures, then B5 measures accuracy on the dataset. Nothing merges to `main` with a failing test.

Test layers

Layer	What it checks	Owner	Where
Unit	Each function against hand-made examples and edge cases (listed in every module above)	Module owner	`tests/test_<module>.py`
Property	Fast version equals slow reference on random input: rolling hash vs naive, deque winnowing vs brute force, union-find vs BFS	Module owner	same files, seeded random
Guarantee	Winnowing finds every shared block of length `w + k - 1` or more, 1,000 trials	A	`tests/test_winnow.py`
Integration	`run()` on a 6-file fixture gives the expected flagged pairs and the expected JSON	Both, in S3	`tests/test_pipeline.py`, `tests/fixtures/`
Regression	Every bug you fix gets a test that would have caught it	Whoever fixes it	next to the module
Evaluation	Precision, recall, F1 on the dataset	B	`python -m zplus eval`

Run `pytest --cov=zplus` and aim for at least 85% line coverage on the engine. Put `pytest` in a GitHub Actions workflow so every push is checked.

Targets for the final evaluation

These are goals to aim at, not results. Replace them with your measured numbers once B5 runs, and report honestly if you miss one.

Measure	Target	Measured on
Recall on level 1 copies (cosmetic)	at least 0.95	Held-out problems 4 and 5
Recall on level 2 copies (light structure)	at least 0.90	Held-out problems 4 and 5
Recall on level 3 copies (structural)	at least 0.70	Held-out problems 4 and 5
Precision at the chosen threshold	at least 0.90	Held-out problems 4 and 5
z-plus recall minus raw-text diff baseline recall, at equal precision	clearly positive	All problems
Time to scan 200 submissions	under 10 seconds	Synthetic set of 200 files

How to read a miss

- **Low recall at level 3:** expected, since some structural disguises change the token sequence. Say which operations beat you and why.
- **Low precision:** look at the false positives. Usually honest solutions that share an approach. Tighten `max_df`, raise `k` or the threshold, and re-measure.
- **Slow run:** profile first. The usual cause is a long posting list in A4 or a large tiling table in B2.

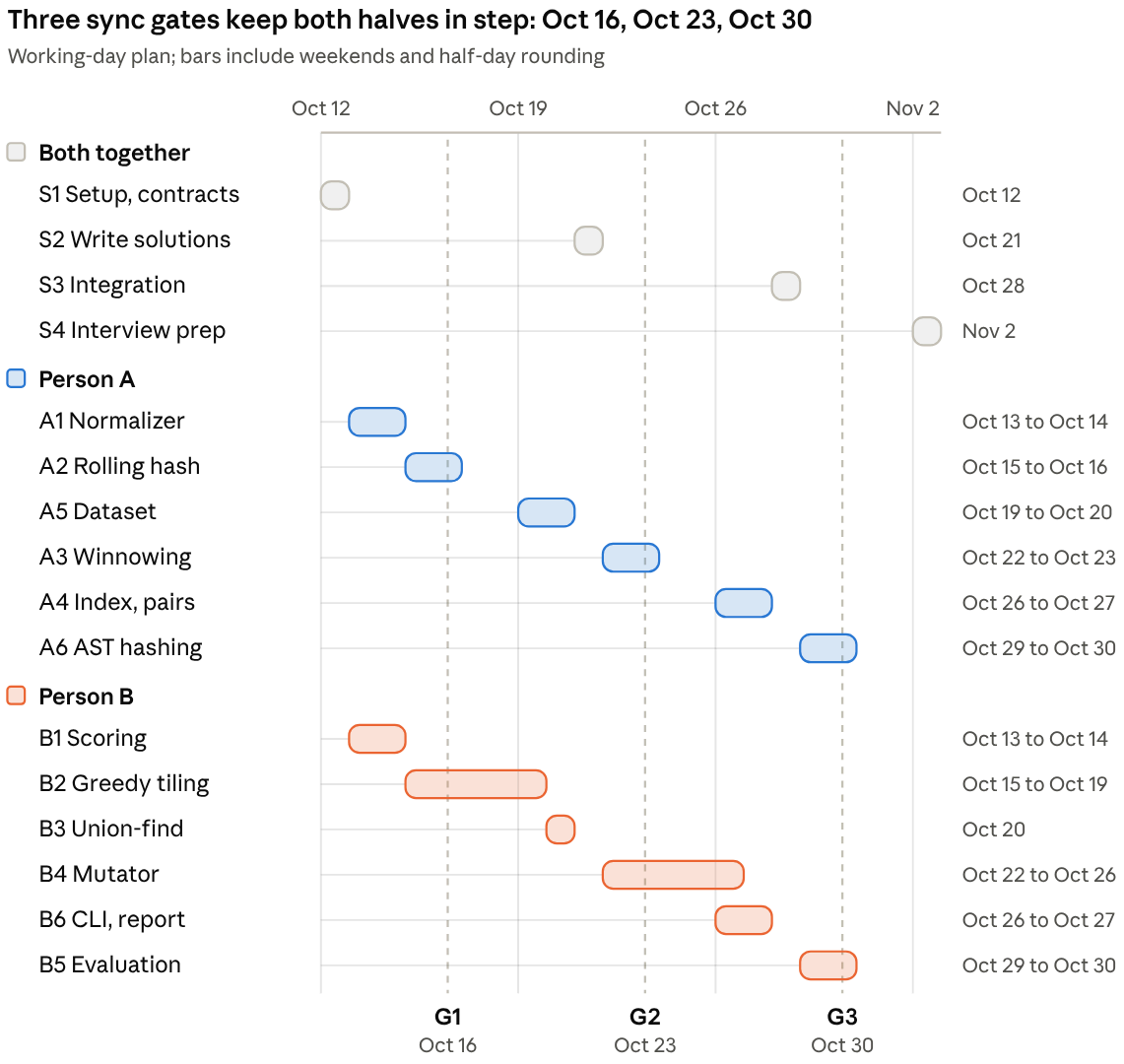
Definition of done for the whole project

- [ ] All unit, property, integration and guarantee tests pass in CI.
- [ ] `python -m zplus scan` produces a text, JSON and HTML report on the dataset.
- [ ] `python -m zplus eval` regenerates every table and plot from scratch.
- [ ] The README has a "How it works" paragraph per module, written in your own words.
- [ ] The README states the measured numbers, the held-out split and the limitations.

- [ ] Each person has explained the other's modules out loud at least once.

Timeline

Both halves run in parallel for 16 working days, from Mon Oct 12 to Mon Nov 2, with three sync gates. The dates assume you start on Monday Oct 12; shift them all if your deadline differs.



parallel plan · 16 working days, 3 sync gates

Each person works down their own lane. At each dashed gate line, everything due so far must be merged and green before either person moves on.

Sync gates

Gate	Date	What must be true
G1	Fri Oct 16	A1 and A2 merged: a file becomes tokens and then k-gram hashes. B1 merged. B2 is in progress on hand-made tokens. Tag `v0.1`.
G2	Fri Oct 23	A5 dataset validated with 40 originals. A3 merged. B3 done. B4 writes copies that compile and pass `check_solution`. Tag `v0.2`.
G3	Fri Oct 30	`run()` works end to end (S3). A6 merged. B5 has produced `results/summary.md`. The CLI report works. Tag `v1.0`.

Week 1 day by day

Day	Person A	Person B
Mon Oct 12	S1 together: repo, `types.py`, empty module stubs, pytest, CI file	S1 together
Tue Oct 13	A1: loader, token mapping table, first tests	B1: Jaccard and containment with tests
Wed Oct 14	A1: f-strings, docstrings, error files, README paragraph	B1: `combine`, README paragraph; start B2's run table
Thu Oct 15	A2: naive hash first, then the rolling version	B2: longest run and the marking loop
Fri Oct 16	A2: property tests; gate review	B2: tiles and coverage score; gate review

Weeks 2 and 3 follow the bars. B5 runs the AST-dependent experiments last, because A6 lands on Oct 29 and 30.

If you fall behind

- Cut in this order: A6 and the AST column first, then the HTML report (keep text and JSON), then faster tiling. The tiling is already optional.

- Never cut: the tests, the dataset with ground truth, and B5′s evaluation. They are what make the project credible.
- Weekends are deliberate buffer; the bars include them.

Collaboration rules

These rules keep two people from blocking each other and make sure both of you can explain the whole project in an interview.

Git workflow

- `main` is always green: tests pass and the CLI runs.
- One branch per module: `a/a3-winnow`, `b/b2-tiling`. No direct commits to `main`.
- One pull request per module. **The other person reviews it.** Reviewing forces each of you to read the half you did not write.
- Merge with squash. Commit messages start with the module id: `A3: add deque-based winnowing`.
- Tag `v0.1` at the end of week 1, `v0.2` at the end of week 2, `v1.0` at the end of week 3.

Pull request checklist

- [ ] Tests for the module pass, including edge cases listed in this document.
- [ ] Every public function has type hints and a docstring with its time complexity.
- [ ] No built-in `hash()` on strings, no `print` in engine code, no global state.
- [ ] The module′s "How it works" paragraph is added to the README, in your own words.
- [ ] The reviewer can explain what the module does without asking the author.

Rhythm

- **Daily, 10 minutes:** what I finished, what blocks me, what is next.
- **Weekly teach-back, 20 minutes:** each person explains their own modules to the other as if in an interview. The listener writes down every question they could not answer and the author fixes the README or the code comments.
- **Sync points** (G1, G2, G3 in the timeline): agree on interface changes in writing before changing `types.py`.

Code standards

- Python 3.10 or newer, type hints everywhere, 88-character lines.
- Run a formatter (`black`) and a linter (`ruff`) before each commit. Both are optional tools, not part of the engine.
- Names: modules in `snake_case` , classes in `CamelCase` , constants in `UPPER_CASE` .
- Randomness only through an explicit `random.Random(seed)` .

Rule about outside help

You may use any tool to understand a concept. You must be able to write each algorithm from a blank page and explain every line. Interviewers often ask for exactly that, and a project you cannot reproduce hurts more than it helps. Do not commit code you cannot explain.

Interview preparation

The project is the vehicle; what you are really showing is that you can pick an algorithm, justify it, implement it, and measure it. Prepare these pieces together in task S4.

The pitch

30 seconds: "We built z-plus, a code plagiarism detector for Python. Renaming variables or reordering lines fools a text diff, so we normalize code into tokens, hash overlapping token windows with a rolling hash, and keep a guaranteed-overlap sample using winnowing. An inverted index finds candidate pairs without comparing every file, greedy string tiling verifies them and shows the matching regions, and union-find groups the copiers. We measured accuracy with our own disguise generator."

Two-minute story, in this order: the problem, why a text diff fails, the pipeline stage by stage, how you tested it, the numbers, what it cannot do.

Complexity cheat sheet

Let L be characters in a file, n tokens, F fingerprints, N files, P the length of a posting list, C candidate pairs.

Stage	Time	Space
A1 normalize	O(L)	O(n)

Stage	Time	Space
A2 rolling hash	O(n)	O(n)
A3 winnowing	O(n)	O(n)
A4 index build	O(total F)	O(total F)
A4 pair counting	O(sum of P squared over hashes)	O(pairs)
A6 AST hashing	O(AST nodes)	O(AST nodes)
B1 scoring	O(F)	O(F)
B2 tiling (per pair)	O(n m) per round	O(n m)
B3 clustering	O(E alpha(N))	O(N)

Whole scan: roughly O(N L) for reading and hashing, plus the pair counting, plus C tiling runs. Only C pairs reach the expensive step, not all N squared.

Likely questions

Question	Answer outline
Why a rolling hash?	Hashing each k-gram from scratch is O(nk). Rolling reuses the previous hash and costs O(1) per step, so O(n) total.
Why not Python's `hash()`?	Randomized per process, so fingerprints change between runs and cannot be compared or cached.
What does winnowing guarantee?	Any shared run of at least `w + k - 1` tokens yields a shared fingerprint; runs shorter than `k` are never reported.
Why winnowing instead of keeping hashes divisible by p?	That rule can pick nothing across a long block, so it has no guarantee.
Why an inverted index?	Comparing all pairs costs O(N squared) file comparisons. The index touches only pairs that already share a fingerprint.

Question	Answer outline
What are the stop-word hashes?	Hashes common to many files (input parsing, boilerplate). They create huge posting lists and false matches, so they are dropped.
Why union-find?	Streaming edges, near-constant time per operation, no adjacency lists. Path compression plus union by size.
How do you handle false positives?	Verification by tiling, a threshold chosen on a tuning split, hard negatives in the dataset, and a report that shows evidence for a human.
How do you know it works?	Our mutation generator makes copies with known labels; we report precision, recall and F1, tuned on three problems and tested on two.
What does it miss?	Different algorithms with the same behavior, heavy restructuring, other languages. The report says what level of disguise it stops.
How would it scale to a million files?	MinHash with locality-sensitive hashing to find candidates, a sharded index, and tiling only on the top few candidates.
Is it fair to accuse someone based on this?	No. It produces leads with evidence. A human decides.

Trade-offs to be ready to defend

- Normalizing all names to `ID` raises recall and lowers precision.
- `max(tiling, ast)` catches more disguises and allows more false positives than an average.
- A small `k` finds short copies and creates noise; a large `k` misses small edits.
- A small `w` keeps more fingerprints (more accurate, slower); a large `w` keeps fewer.

Limitations and future work

- Other languages through a generic parser such as tree-sitter.

- Semantic clones (different algorithm, same output) would need program analysis or learned embeddings.
- Running Karp-Rabin GST for faster tiling.
- MinHash and LSH for very large collections.
- Behavioral signals such as typing speed and paste events, which is the original anti-cheat direction. A small web front end for reviewers.

Numbers to write down after B5 runs

Keep these on one index card.

- [ ] Dataset size: files, problems, positive and negative pairs.
- [ ] Chosen `k`, `w`, threshold, and why.
- [ ] Recall at each mutation level, on the held-out problems.
- [ ] Precision and F1 at the chosen threshold.
- [ ] Improvement over the raw-text baseline.
- [ ] Scan time for 200 files.
- [ ] One false positive and one miss, and what each taught you.